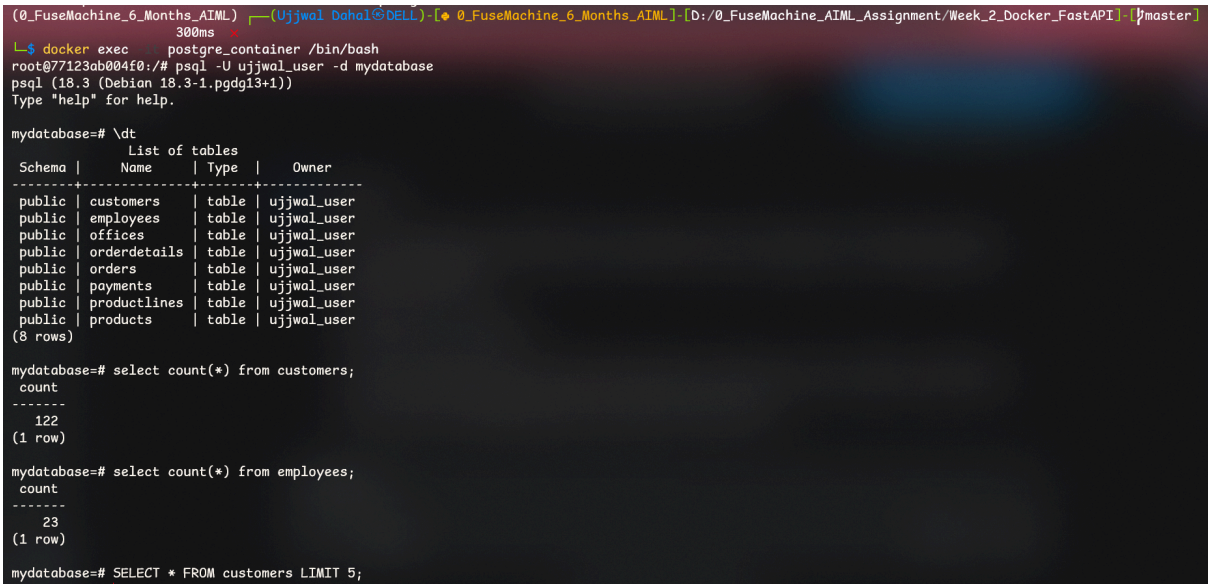
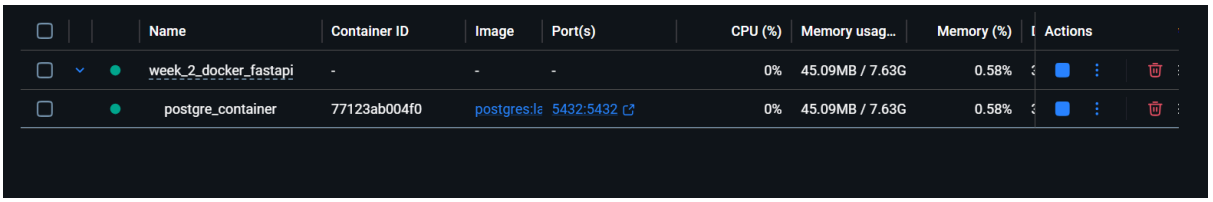
Week 2 Assignment

Task 1: PostgreSQL Setup with Docker





Q1. Why is .env better than writing passwords in code?

-> .env file is better because it keeps usernames and passwords separate from the main code.

This improves security and makes the project easier to manage. Similarly sensitive data can be kept inside the .env file so that later on we cannot push it into GitHUB for security purposes by including the .env file in the .gitignore file.

Q2. Why is treating the database as a separate service useful?

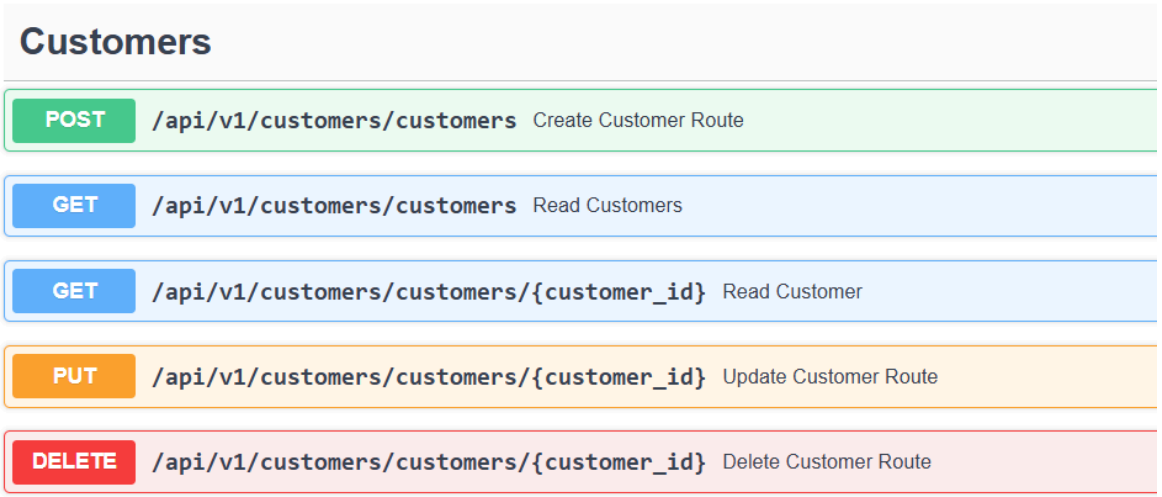
-> Treating the database as a separate service is useful because the application and database can work independently. The database can be updated or restarted without changing the application. This makes the system easier to maintain and scale.

Q3. Why does Docker make development and production similar?

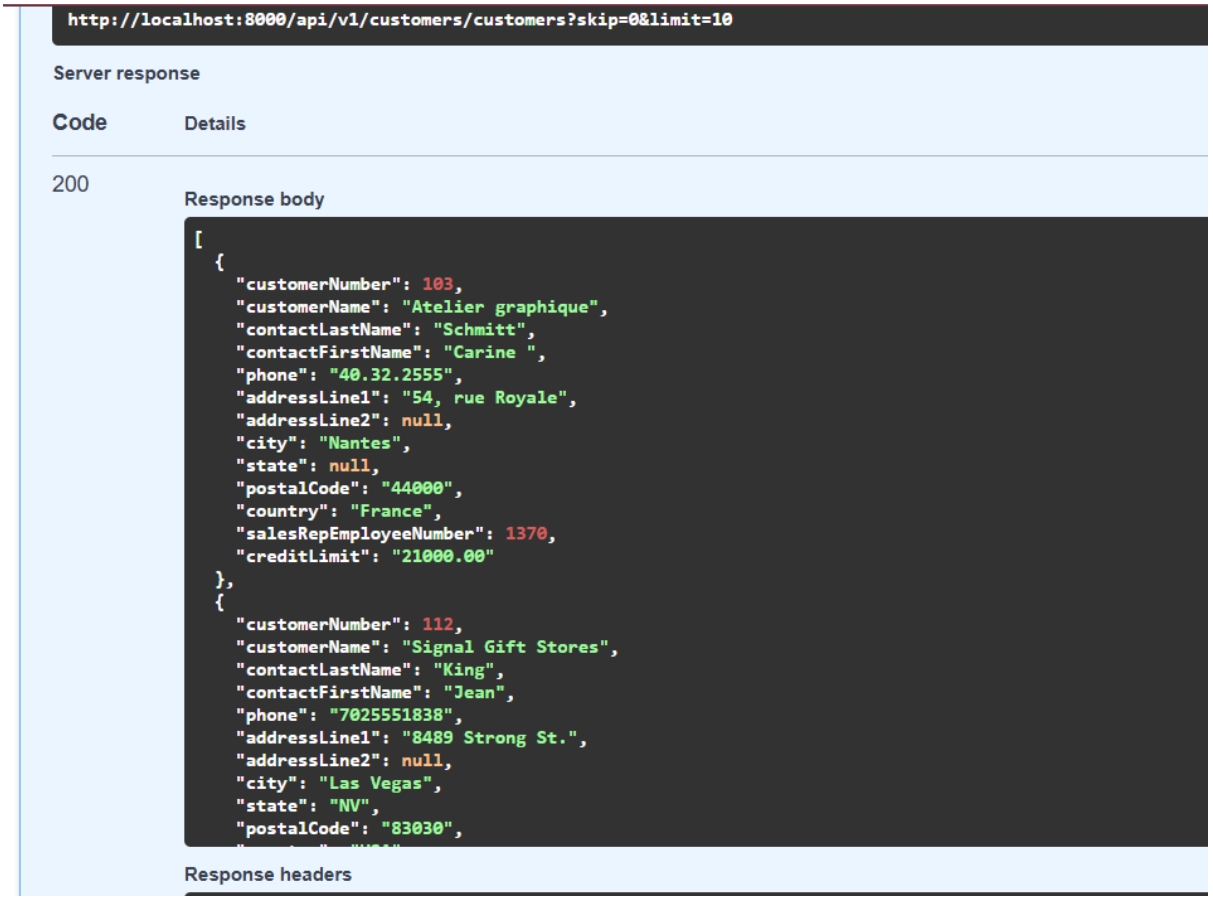
-> Docker makes development and production similar by creating the same environment everywhere.

The same software, settings, and dependencies are used on all systems. This reduces environment-related problems and helps the application run consistently.

Task 2: Building a Customer API



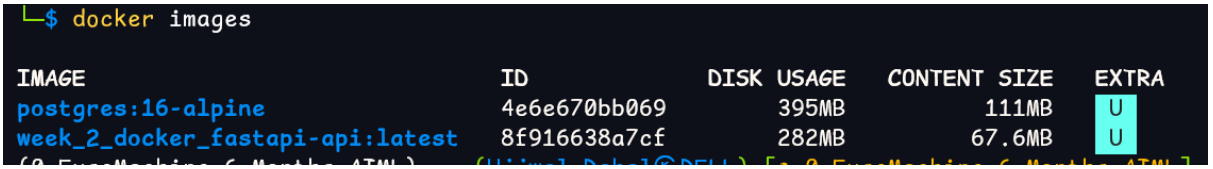
Get Request Response



Containers



Images



Task 3: Concurrency

Concurrency Proof

The logs clearly show that the /overall_counts endpoint is using concurrent execution instead of sequential processing.

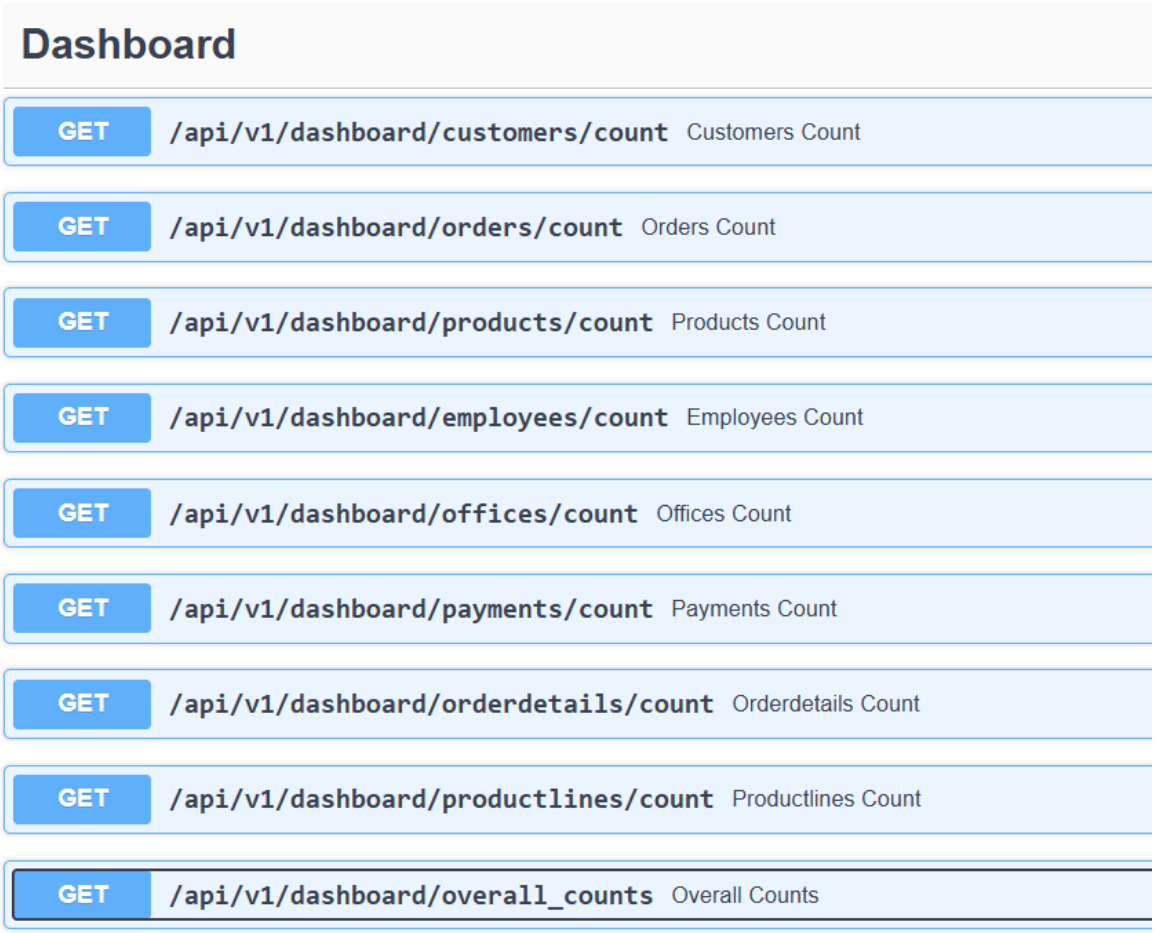
When the request is received, the system immediately starts the database session and triggers all count operations at almost the same time. You can see that all queries like customers, orders, products, employees, offices, payments, orderdetails, and productlines start within a few milliseconds of each other.

This is important because it proves that the system is not waiting for one query to finish before starting the next one. Instead, all database counting operations are running in parallel using asyncio.gather().

Finally, all tasks complete together, and the total response time is very fast at around 0.0277 seconds, which confirms high performance and efficient concurrent processing. The database session is then properly closed after completing the work.

```
2026-05-10 18:02:51,676 - INFO - Database session opened
2026-05-10 18:02:51,676 - INFO - GET /overall_counts called
2026-05-10 18:02:51,676 - INFO - Starting concurrent dashboard queries
2026-05-10 18:02:51,681 - INFO - Counting customers
2026-05-10 18:02:51,682 - INFO - Counting orders
2026-05-10 18:02:51,683 - INFO - Counting products
2026-05-10 18:02:51,683 - INFO - Counting employees
2026-05-10 18:02:51,684 - INFO - Counting offices
2026-05-10 18:02:51,686 - INFO - Counting payments
2026-05-10 18:02:51,686 - INFO - Counting orderdetails
2026-05-10 18:02:51,689 - INFO - Counting productlines
2026-05-10 18:02:51,703 - INFO - All concurrent queries completed
2026-05-10 18:02:51,704 - INFO - Dashboard completed in 0.0277 seconds
2026-05-10 18:02:51,706 - INFO - Database session closed
```

Dashboard API



This is Dashboard API Created using FastAPI

Overall Counts Output

Request URL

```
http://localhost:8000/api/v1/dashboard/overall_counts
```

Server response

Code	Details
200	Response body


```
{
  "customers": 122,
  "orders": 326,
  "products": 110,
  "employees": 23,
  "offices": 7,
  "payments": 273,
  "orderdetails": 2996,
  "productlines": 7
}
```


Response headers

Task 4: API of All Tables

Customers

POST	/api/v1/customers/customers	Create Customer Route
GET	/api/v1/customers/customers	Read Customers
GET	/api/v1/customers/customers/{customer_id}	Read Customer
PUT	/api/v1/customers/customers/{customer_id}	Update Customer Route
DELETE	/api/v1/customers/customers/{customer_id}	Delete Customer Route

Products

GET	/api/v1/products/products	Read Products
POST	/api/v1/products/products	Create Product Route
GET	/api/v1/products/products/{product_code}	Read Product
PUT	/api/v1/products/products/{product_code}	Update Product Route
DELETE	/api/v1/products/products/{product_code}	Delete Product Route

Employees

GET	/api/v1/employees/employees	Read Employees
POST	/api/v1/employees/employees	Create Employee Route
GET	/api/v1/employees/employees/{employee_id}	Read Employee
PUT	/api/v1/employees/employees/{employee_id}	Update Employee Route
DELETE	/api/v1/employees/employees/{employee_id}	Delete Employee Route

Offices

GET	/api/v1/offices/offices	Read Offices
POST	/api/v1/offices/offices	Create Office Route
GET	/api/v1/offices/offices/{office_code}	Read Office
PUT	/api/v1/offices/offices/{office_code}	Update Office Route
DELETE	/api/v1/offices/offices/{office_code}	Delete Office Route

ProductLines

GET	/api/v1/productlines/	Read Productlines
POST	/api/v1/productlines/	Create Productline Route
GET	/api/v1/productlines/{product_line}	Read Productline
PUT	/api/v1/productlines/{product_line}	Update Productline Route
DELETE	/api/v1/productlines/{product_line}	Delete Productline Route

Orders

GET	/api/v1/orders/orders	Read Orders
POST	/api/v1/orders/orders	Create Order Route
GET	/api/v1/orders/orders/{order_id}	Read Order
PUT	/api/v1/orders/orders/{order_id}	Update Order Route
DELETE	/api/v1/orders/orders/{order_id}	Delete Order Route

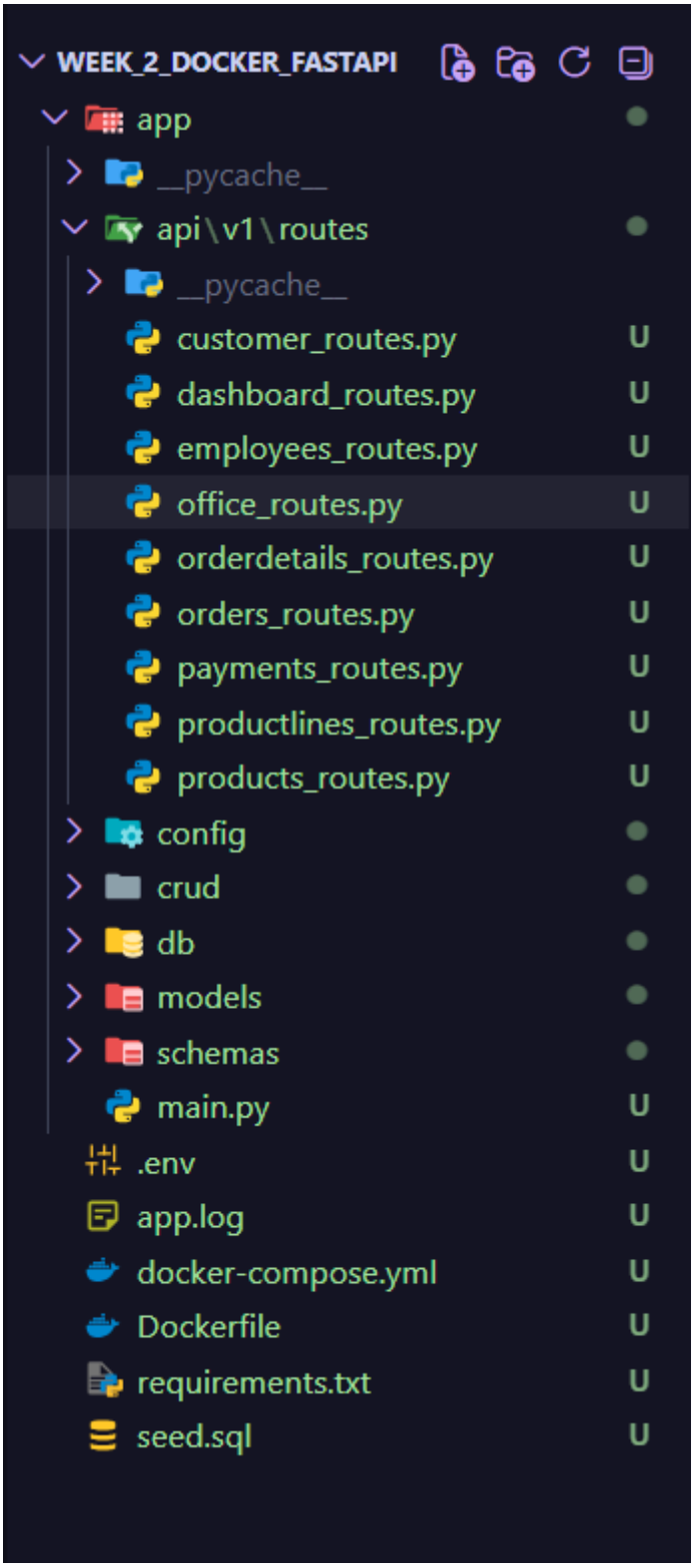
OrderDetails

GET	/api/v1/orderdetails/orderdetails	Read Orderdetails
POST	/api/v1/orderdetails/orderdetails	Create Orderdetail Route
GET	/api/v1/orderdetails/orderdetails/{order_id}/{product_code}	Read Orderdetail
PUT	/api/v1/orderdetails/orderdetails/{order_id}/{product_code}	Update Orderdetail Route
DELETE	/api/v1/orderdetails/orderdetails/{order_id}/{product_code}	Delete Orderdetail Route

Payments

GET	/api/v1/payments/payments	Read Payments
POST	/api/v1/payments/payments	Create Payment Route
GET	/api/v1/payments/payments/{customer_id}/{check_number}	Read Payment
PUT	/api/v1/payments/payments/{customer_id}/{check_number}	Update Payment Route
DELETE	/api/v1/payments/payments/{customer_id}/{check_number}	Delete Payment Route

Folder Structure



Payments Get Request Response

Request URL

```
http://localhost:8000/api/v1/payments/payments?skip=0&limit=10
```

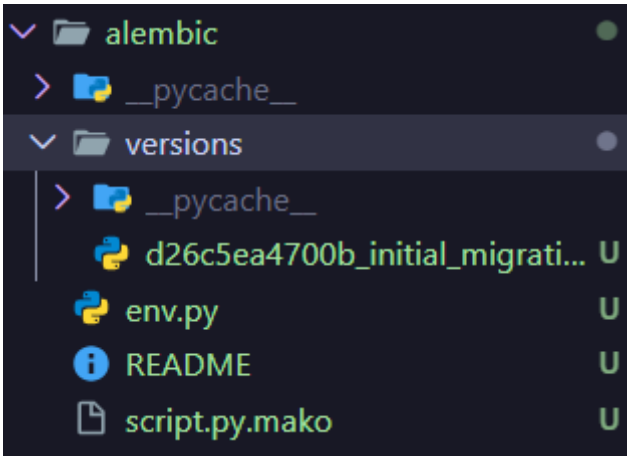
Server response

Code	Details
200	Response body


```
[
  {
    "paymentDate": "2004-10-19",
    "amount": 6066.78,
    "customerNumber": 103,
    "checkNumber": "HQ336336"
  },
  {
    "paymentDate": "2003-06-05",
    "amount": 14571.44,
    "customerNumber": 103,
    "checkNumber": "JM555205"
  },
  {
    "paymentDate": "2004-12-18",
    "amount": 1676.14,
    "customerNumber": 103,
    "checkNumber": "OM314933"
  },
  {
    "paymentDate": "2004-12-17",
    "amount": 14191.12,
    "customerNumber": 112,
    "checkNumber": "BO864823"
  },
  {
    "paymentDate": "2003-06-06",
```

Adding Migration using Alembic

```
└─$ docker exec -it fastapi_container bash
root@ba53453c0d4e:/app# alembic --version
alembic 1.18.4
root@ba53453c0d4e:/app# alembic revision --autogenerate -m "initial migration"
INFO  [alembic.runtime.migration] Context impl PostgresqlImpl.
INFO  [alembic.runtime.migration] Will assume transactional DDL.
INFO  [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.schemas
INFO  [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.tables
INFO  [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.types
INFO  [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.constraints
INFO  [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.defaults
INFO  [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.comments
  Generating /app/alembic/versions/d26c5ea4700b_initial_migration.py ...  done
root@ba53453c0d4e:/app# alembic upgrade head
INFO  [alembic.runtime.migration] Context impl PostgresqlImpl.
INFO  [alembic.runtime.migration] Will assume transactional DDL.
INFO  [alembic.runtime.migration] Running upgrade  -> d26c5ea4700b, initial migration
root@ba53453c0d4e:/app#
```



Journal

While setting up PostgreSQL with Docker Compose, I first used:

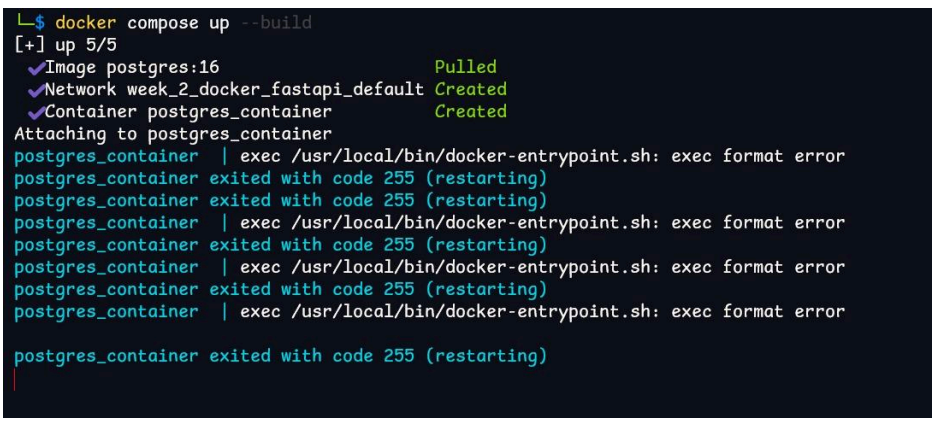
```yaml
image: postgres:16
```

But when I ran:

```bash
docker compose up --build
```

I got this error:

```bash
exec /usr/local/bin/docker-entrypoint.sh: exec format error
```



The PostgreSQL container kept restarting with exit code 255.

After troubleshooting, I changed the image to the Alpine version:

```yaml
image: postgres:16-alpine
```

Then the container started successfully and the problem was solved.

What I Learned

- postgres:16 caused an architecture/compatibility issue on my system.
- postgres:16-alpine is a lightweight Alpine Linux based image.
- Alpine images are smaller and sometimes work better depending on the environment/system compatibility.
- Docker image versions can behave differently, so trying another variant like -alpine can help fix issues.

I learned Docker properly, including how to containerize applications and manage services using Docker Compose.

I also learned how to build Backend APIs using FastAPI.
During this learning process, I understood:

- Creating APIs with FastAPI
- Routing and CRUD operations
- Database connection setup
- Using SQLAlchemy ORM
- Environment variables with `.env`
- Dockerizing FastAPI applications
- Connecting FastAPI with PostgreSQL using Docker

Additionally, I learned Database Migration using Alembic.